qual_summary.py 사용법과 설명

lab11/src/qual_summary.py는 FASTQ 파일을 읽어 전체 read의 평균 품질 점수(Phred)를 출력하는 스크립트이다. 표준 라이브러리(sys)만 사용한다. FASTQ를 읽는 read_fastq 함수는 parse_fastq.py의 것과 같다.

1. 사용법

lab 폴더(lab11/)에서 FASTQ 파일 경로를 인자로 주고 실행한다.

```
cd lab11
python3 src/qual_summary.py data/SRR1553607_1.fastq
```

인자	설명
sys.argv[1]	읽을 FASTQ 파일 경로 (필수)

실행 결과 예시

```
$ python3 src/qual_summary.py data/SRR1553607_1.fastq
전체 read 평균 품질: 33.4497222958299

$ python3 src/qual_summary.py data/SRR1553607_2.fastq
전체 read 평균 품질: 30.830840894308746
```

두 파일 모두 203,445개의 read가 들어 있고, 모든 read의 길이는 101이다. paired-end의 두 번째 파일(_2)이 첫 번째 파일(_1)보다 평균 품질이 약 2.6 낮다. Illumina paired-end에서 read 2의 품질이 더 낮은 것은 흔한 현상이다.

2. Phred 품질 점수

FASTQ의 4번째 줄은 염기마다 품질 문자 하나를 가진다. 품질 문자는 Phred+33 인코딩이므로, 문자의 ASCII 값에서 33을 빼면 Phred 점수 Q가 된다.

문자	ASCII 값	Q (ASCII - 33)	오류 확률 (10^(-Q/10))
#	35	2	약 63%
+	43	10	10% (1/10)
5	53	20	1% (1/100)
?	63	30	0.1% (1/1,000)
I	73	40	0.01% (1/10,000)
J	74	41	약 0.008%

일반적으로 Q30 이상이면 품질이 좋다고 본다. 실습 데이터의 점수 범위는 2~41이다.

3. 코드 설명

read_fastq(filename)

FASTQ 파일을 읽어 (이름, 서열, 품질) 튜플의 리스트를 반환한다.

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()          # 파일 전체를 줄 단위 리스트로 읽음
    for i in range(0, len(lines), 4):  # 4줄씩 건너뛰며 read 하나씩 처리
        name = lines[i].rstrip()[1:]   # 1줄: 맨 앞 '@' 제거
        seq = lines[i + 1].rstrip()    # 2줄: 서열
        qual = lines[i + 3].rstrip()   # 4줄: 품질 (3줄 '+'는 사용하지 않음)
        records.append((name, seq, qual))
    return records
```

항목	내용
입력	FASTQ 파일 경로 (문자열)
반환값	[(name, seq, qual), ...] 형태의 리스트

mean_quality(qual)

품질 문자열 하나를 받아 그 read의 평균 Phred 점수를 반환한다.

```
def mean_quality(qual):
    scores = []
    for ch in qual:
        scores.append(ord(ch) - 33)    # 문자 -> ASCII 값 -> Phred 점수
    return sum(scores) / len(scores)   # 점수의 평균
```

항목	내용
입력	품질 문자열 (예: "BB@FFFFF...")
반환값	평균 Phred 점수 (실수)
예	mean_quality("II##") → (40 + 40 + 2 + 2) / 4 = 21.0

ord(ch)는 문자의 ASCII 코드 값을 돌려주는 내장 함수이다. 예를 들어 ord("I")는 73이다.

메인 부분

```
reads = read_fastq(sys.argv[1])

total = 0
for name, seq, qual in reads:
    q = mean_quality(qual)     # read 하나의 평균 품질
    total = total + q          # 누적

print("전체 read 평균 품질:", total / len(reads))
```

1. 명령줄 인자로 받은 FASTQ 파일을 읽는다.
2. read마다 mean_quality로 평균 품질을 구해 total에 더한다.
3. total을 read 개수로 나누어 read별 평균 품질의 평균을 출력한다.

4. 결과 해석: read 평균의 평균

이 스크립트가 출력하는 값은 "모든 염기의 평균 품질"이 아니라 "read별 평균 품질을 다시 평균한 값"이다. read 길이가 모두 같으면 두 값은 같지만, 길이가 다르면 달라진다.

```
@a
ACGT
+
IIII        <- 4개 염기, 모두 Q40 -> read 평균 40
@b
AC
+
##          <- 2개 염기, 모두 Q2  -> read 평균 2
```

계산 방법	식	결과
read 평균의 평균 (이 스크립트)	(40 + 2) / 2	21.0
전체 염기의 평균	(40×4 + 2×2) / 6	27.33

실습 데이터(SRR1553607)는 모든 read가 101bp로 같으므로 두 값이 일치한다. 다만 fastp 등으로 trimming한 뒤에는 read 길이가 달라지므로 두 값의 차이에 주의한다.

5. 주의사항과 한계

- 메모리 사용: readlines()는 파일 전체를 메모리에 올린다. 실습 데이터(약 20만 read)에는 문제가 없지만 수 GB 크기의 FASTQ에는 한 줄씩 읽는 방식이 적합하다.
- 형식 검사 없음: 모든 read가 정확히 4줄이라고 가정한다. 줄 수가 4의 배수가 아니면 IndexError가 난다.
- 인자 누락: 파일 경로 없이 실행하면 sys.argv[1]에서 IndexError가 난다.
- 빈 파일: read가 하나도 없으면 total / len(reads)에서 ZeroDivisionError가 난다.
- 빈 품질 줄: 길이 0인 read가 있으면 mean_quality에서 ZeroDivisionError가 난다.
- 인코딩 가정: Phred+33(Illumina 1.8 이후, Sanger)만 처리한다. 오래된 Phred+64 데이터에서는 점수가 31만큼 높게 계산된다.
- 압축 파일 미지원: .fastq.gz는 직접 읽을 수 없으므로 gunzip으로 먼저 압축을 풀어야 한다.

6. 활용 예

mean_quality를 이용하면 품질 기준으로 read를 걸러낼 수 있다. 다음은 평균 품질이 Q30 이상인 read의 개수와 비율을 세는 예이다.

```
good = 0
for name, seq, qual in reads:
    if mean_quality(qual) >= 30:
        good = good + 1
```

```
print("Q30 이상 read:", good, "/", len(reads))
print("비율(%):", good / len(reads) * 100)
```

같은 작업은 seqkit fx2tab -q로 read별 평균 품질을 출력하거나, fastp/FastQC 보고서에서도 확인할 수 있다.